COMP3438 System Programming

Background for System Programs

Definition

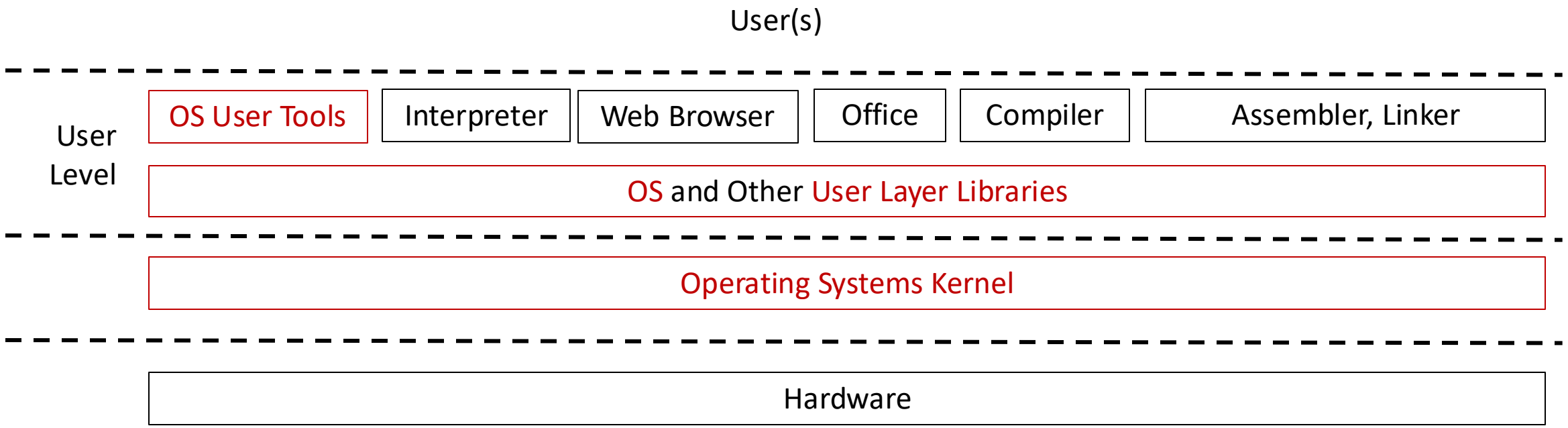
An operating system is an intermediary between a computer user and the hardware

Make the hardware convenient to use

Manages system resources

Use the hardware in an efficient manner

Runtime View of Operating Systems



Types of OSs

Batch

- Single CPU
- User submits large number of tasks at one time
- OS decides what to run and when
- Back-to-back single tasking

Time-Sharing

- Multiple users connected to a single CPU
- Many user terminals
- Multiple tasks run simultaneously using time-sharing, giving users the feeling of having multiple dedicated CPUs running in parallel

Types of OSs

Parallel

- Multiple CPUs closely coupled to form one computer
- Higher throughput and better fault tolerance
- Each CPU can be running batch tasking or time-sharing multitasking

Distributed

- Multiple CPUs loosely coupled via networking

Real-time

- Very strict response time requirements
- Periodical tasks, every job of a task has a strict deadline

Single Tasking System

Each task, once started, executes to its very end without any interruption from other task(s).

Simple to implement: sequential boundaries between tasks, and resource accesses.

Few security risks

Poor utilization of the CPU and other resources

Example: MS-DOS

Multitasking System

Very complex

Serious security issues

- How to protect one task from another, which shares the same block of memory

Much higher utilization of system resources

Support interactive user/physical-world interface

Example: Unix, Windows NT

Hardware Basics

OS and hardware closely tied together

Basic hardware resources:

- CPU
- Memory
- Disk
- I/O

CPU

CPU controls everything in the system

- It is involved in any work needs to be done

Most precious resource

- This is what you are paying for
- Users would usually prefer high utilization (from useful work)

Only one process running on a CPU at a time*

Millions or even billions of machine instructions per second

- Getting faster all the time

Memory

Limited capacity

- Never enough, and expanding over the years

Temporary (volatile) storage

Electronic storage

- Fast, random access

Any process to run on the CPU must first be loaded into memory

I/O

Many I/O devices

- Keyboard, mouse, monitor, printer etc.

Most I/O devices are painfully slow

Need to find ways to deal with high I/O latency

- Like multiprogramming

Protection and Security

OS must protect itself from users

- Reserved memory only accessible by OS

OS may protect users from one another

Interrupts

Modern OSs are event driven

Event is signaled by special hardware signal sent to the CPU

Two types of events

- Hardware Interrupts: caused by external hardware, can occur at anytime
- Software Interrupts (aka Exceptions, Traps): caused by software, synchronous to CPU clock

Interrupt Philosophy

Use an interrupt table and special hardware to redirect CPU execution

By the end of interrupt handling, the CPU can resume the interrupted process

Interrupt Table

Large array of addresses indicating what code to run (interrupt routine) for a given interrupt

Each interrupt has a corresponding number associated with it

- On Intel processors this is from 0 to 255
- This gives fixed size interrupt table

Use the interrupt number to index into the array to find out what code to run (interrupt routine)

Hardware to Support Hardware Interrupt

Programmable Interrupt Controller (PIC)

Connected to I/O devices via Interrupt Request Lines (IRQ) or

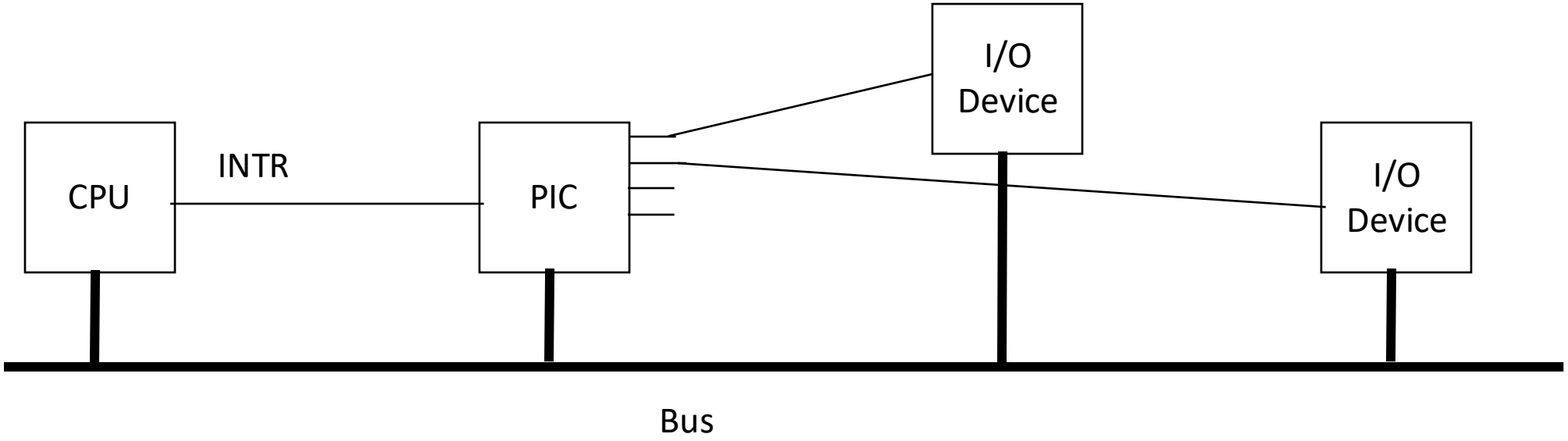
Bus (e.g. PCI Express)

PIC connected to the CPU by a special line or bus

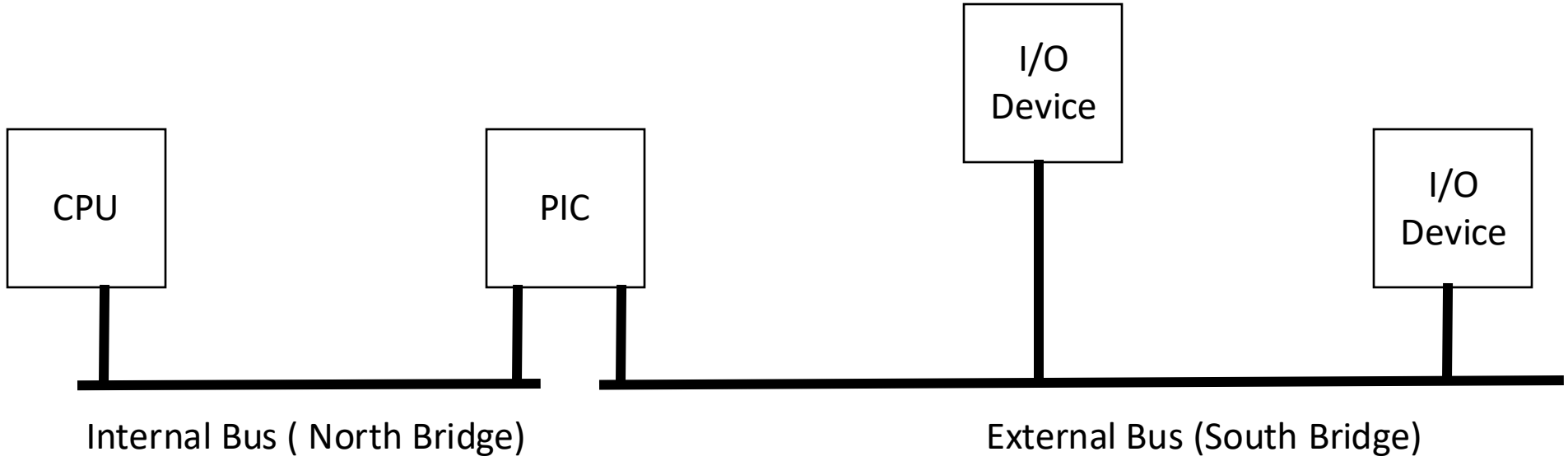
CPU <==> PIC <==> I/O devices

<==> can be special signals via dedicated wire lines (IRQ), or special sequences of back-and-forth messages via bus(es) (virtual IRQ)

IRQ System Architecture (Conventional)



IRQ System Architecture (Example of Message Passing Virtual IRQ)



Hardware Handling of Interrupts (Conventional IRQ as an Example)

After each instruction executes, CPU checks if the IRQ pin voltage has been raised

If so

1. Sets the system into kernel mode (if not already there)
2. Determine interrupt number (from PIC or instruction)
3. Read appropriate interrupt table entry
   Special register contains base address of interrupt table
   Each entry in table is fixed size so easy to calculate where to look in memory (e.g. `memloc = iptr + 8 * intNum`)
4. Saves the process state to the stack (particularly, the program counter)
5. Saves error code to stack (if it exists)
6. Loads the program counter with the value stored in the interrupt table
   This starts the CPU executing in the interrupt routine

Peripheral Devices

Input Devices

Output Devices

Storage (memory is usually not considered as a peripheral)

Peripheral Devices for Embedded Systems

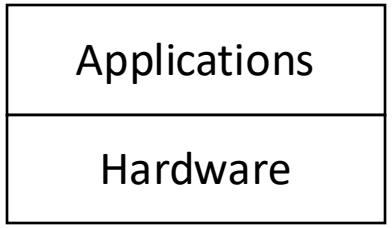
Embedded Systems: computer systems that do not look like conventional computer systems, e.g. mobile devices, devices tightly coupled with the physical world (i.e. cyber-physical systems).

Input Devices: Sensors

Output Devices: Actuators

Historical Perspective

Application built on top of hardware



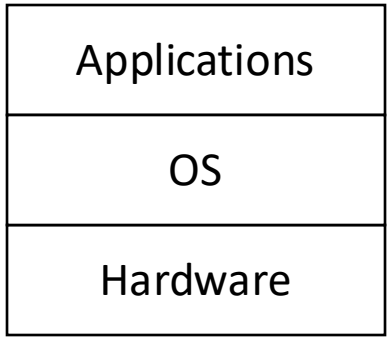
Problems:

Complexity: have to know the tedious low-level programming details of the hardware

Inflexibility: when the hardware changes, the application has to change with it.

Historical Perspective

Application built on top of OS



Ideal:

OS application programming interface (API) hides the complexity and heterogeneity of hardware programming.

As long as the OS API remain the same, the hardware can change.

Design goal conflict:

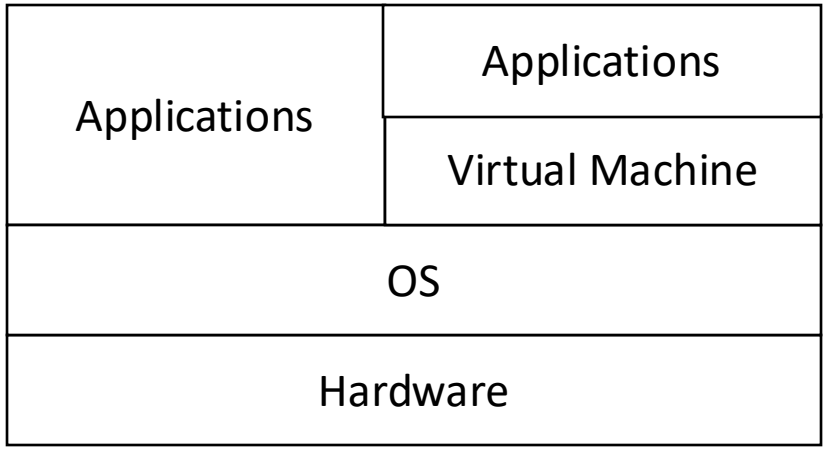
Full exploitation of hardware features

Versus

Full platform independence and simple API

Historical Perspective

Application built on top of OS



Want to exploit more features of the hardware: Application + OS

Want to be more platform independent and simpler API: Application + Virtual Machine (VM)

Ideal:

OS application programming interface (API) hides the complexity and heterogeneity of hardware programming.

As long as the OS API remain the same, the hardware can change.

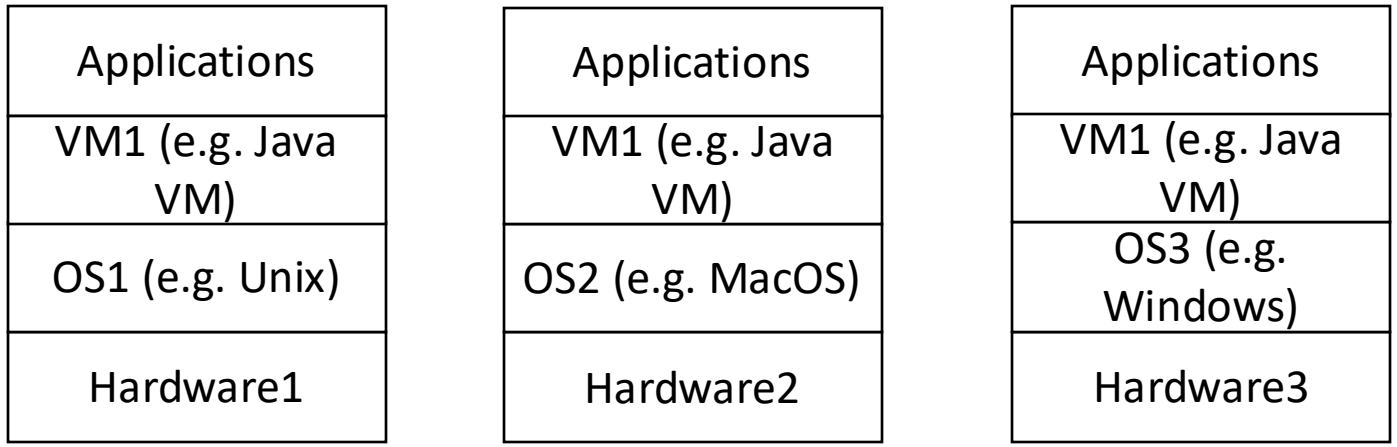
Design goal conflict:

Full exploitation of hardware features

Versus

Full platform independence and simple API

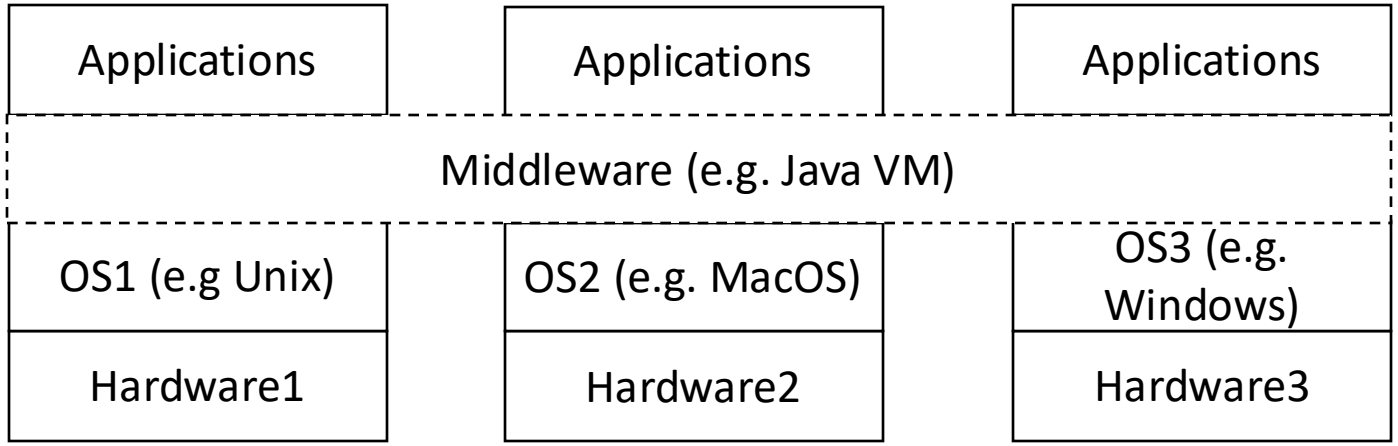
Historical Perspective



Virtual Machine (VM) can hide the heterogeneity of OS APIs

As long as the VM API remain the same, the OS (and hardware) can change, and the application can remain the same.

Historical Perspective



Middleware API (VMs can also be regarded as a kind of middleware) hides the heterogeneity of OS APIs

As long as the Middleware API remain the same, the OS (and hardware) can change, and the application can remain the same.